

Sujet d'examen UE NFE204

Année universitaire 2018-2019

Examen seconde session FOAD

Consignes

Durée de l'examen : 3h00 ; Documents non autorisés ; Calculatrice autorisée

Les exercices sont indépendants les uns des autres. Merci de soigner votre écriture.

Ce sujet contient 4 pages.

Important : nous n'évaluons pas les réponses par la syntaxe. Ne vous inquiétez pas d'utiliser un point-virgule à la place d'un point d'exclamation ou autres détails mineurs

Première partie : modélisation semi-structurée (6 pts)

Un grand magasin enregistre tous les tickets de caisse des transactions effectuées par ses clients. Un ticket de caisse comprend la date et l'heure de chaque transaction, le numéro de la caisse, le prix total, et la liste des articles achetés avec, pour chacun, la quantité et le prix unitaire. Le ticket de caisse comprend également l'identifiant du client si ce dernier a utilisé sa carte de fidélité. Sinon cet identifiant est à null.

1. Dans une base relationnelle, combien de tables faudrait-il pour représenter les informations d'un ticket de caisse (expliquer et donner le schéma de ces tables) ; combien d'insertions de ligne faut-il faire pour chaque ticket ?
2. Si on choisit une base NoSQL, comment représenter un ticket de caisse en JSON. Donnez un exemple simple. Combien d'insertions faut-il faire ?
3. Je veux compter le nombre de transactions par caisse et par jour. Quelle est la requête SQL si les données sont dans une base relationnelle ?
4. Même question si la base est NoSQL : comment calculer le nombre de transactions par caisse et par jour
5. Indiquez les avantages et inconvénients de chaque solution, relationnelle ou NoSQL.

Solution :

1. En relationnel, il faut 2 tables, une pour les tickets avec les champs `noCaisse`, `date`, `heure` et `prixTotal`, une autre pour les articles avec `idArticle`, `quantité` et `prixUnitaire`. Un champ `idClient` dans la table `Ticket` peut être à null. La table `Article` comprend une clé étrangère `idTicket` pour savoir de quel ticket l'achat de l'article fait partie.

Il faut une insertion dans la table `Ticket`, et autant d'insertions dans la table `Article` qu'il y a d'articles achetés dans une même transaction.

2. En JSON, par exemple :

```
{
  "_id": 978,
  "noCaisse": 12,
  "date": "01/01/2017",
  "heure": "13:30",
```

```

    "prixTotal": 58.47,
    "idClient": null,
    "articles": [
      {"idArticle": 1982, "quantité": 5, "prix": 10.45},
      {"idArticle": 34, "quantité": 1, "prix": 45},
      {"idArticle": 154, "quantité": 3, "prix": 3.02},
    ]
  }
}

```

Une insertion suffit.

3. En SQL :

```

select count(*) from Ticket t, Article a
where a.idTicket = t.id
group by date, noCaisse

```

4. En NoSQL, il faut écrire et exécuter un programme MapReduce.

La fonction de Map émet une paire intermédiaire dont la clé est le critère de regroupement, soit la paire (date, numéro de caisse).

```

function fonctionMap ($ticket)
{
  emit ( [$ticket.date, $ticket.noCaisse], 1)
}

```

La fonction de Reduce effectue le compte des paires intermédiaires, autrement dit des tickets ayant même date et même numéro de caisse.

```

function fonctionReduce ($clé, $valeurs)
{
  return ($clé, count($valeurs))
}

```

Deuxième partie : MapReduce (4 pts)

Je veux calculer le montant moyen des tickets par client, pour les clients dont je connais l'identifiant.

1. Décrivez la modélisation MapReduce de ce calcul. Donnez le pseudo-code de chaque fonction, ou indiquez par un texte clair son déroulement.
2. Donnez la forme de la chaîne de traitement (*workflow*), avec des opérateurs Pig ou Spark, qui implante ce calcul.

Solution :

1. La fonction de Map ne prend en compte que les tickets ayant un numéro de client non nul, et émet une paire intermédiaire dont la clé est le critère de regroupement, soit le numéro de client, et la valeur le prix total du ticket.

```

function fonctionMap ($ticket)
{
  if ($ticket.idClient not null) then
    emit ( $ticket.idClient, $ticket.prixTotal)
  end if
}

```

La fonction de Reduce effectue la moyenne des prix.

```
function fonctionReduce ($idClient, $listePrix)
{
    return ($idClient, avg($listePrix))
}
```

2. En Pig :

```
tickets_client = filter tickets BY idClient not null;
tickets_groupes = group tickets_client by idClient;
avg_par_client = foreach tickets_groupes generate group, AVG(tickets_groupes.prixTotal)
```

Troisième partie : recherche d'information (4 pts)

Les articles sont décrits par des mots-clé. Voici les mots-clé de 4 articles a_1 , a_2 , a_3 , a_4 .

1. laitage, céréale, fruit, légume.
2. céréale, viande, protéine, sucre, gras
3. laitage, fruit, sucre, amidon, vitamine
4. céréale, viande

1. Quel est l'idf de "céréale" ? Quelle interprétation donnez-vous à ce résultat ?

Solution : L'idf est proche de 0 ($\log(4/3) = 0,12$), ce qui signifie que le terme est très courant et peu discriminant pour une recherche.

2. Une recherche avec le mot "viande" ramènera quels documents, et dans quel ordre (pas la peine de faire des calculs).

Solution : Le résultat est, dans l'ordre, a_4 puis a_2 . Les deux parlent de viande, mais l'importance relative du terme est plus grande pour a_4 . Pour le calcul, la norme de a_4 est plus petite, et la division par la norme donnera un résultat plus grand.

3. Quelle est la similarité cosinus entre les articles a_1 et a_3 ?

Solution : La norme de a_1 est $\sqrt{4} = 2$. La norme de a_3 est $\sqrt{5}$. Le cosinus est

$$\frac{1 + 1}{2 \times \sqrt{5}} = \frac{1}{\sqrt{5}}$$

Quatrième partie : systèmes distribués (2 pts)

Un groupe de serveurs en mode maître-esclave comprend 5 nœuds. Suite à une fragmentation du réseau, le maître se retrouve avec un seul esclave. Expliquez ce qui se passe alors.

Cinquième partie : questions de cours (4 pts)

1. Donnez une définition de la scalabilité.
2. Définissez le partitionnement d'une collection dans un système distribué. Quel est l'intérêt ?
3. Pour un traitement, est-il en général plus efficace de lire les données sur le disque local ou dans la mémoire RAM d'un serveur de la même armoire ?
4. Quelles sont les opérations proposées par une ressource dans une architecture RESTful ?